```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
iris = load_iris()
```

```
data = pd.DataFrame(
    data=iris.data,
    columns=iris.feature_names
)
```

```
data["target"] = iris.target
```

```
data.head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

Next steps: Generate code with data | New interactive sheet

```
data.shape
```

```
(150, 5)
```

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   sepal length (cm)  150 non-null    float64
 1   sepal width (cm)   150 non-null    float64
 2   petal length (cm)  150 non-null    float64
 3   petal width (cm)   150 non-null    float64
 4   target             150 non-null    int64
dtypes: float64(4), int64(1)
memory usage: 6.0 KB
```

```
X = data.drop("target", axis=1)
y = data["target"]
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)
```

```
linear_svm = SVC(kernel='linear')
linear_svm.fit(X_train, y_train)
```

```
SVC
SVC(kernel='linear')
```

```
y_pred_linear = linear_svm.predict(X_test)
```

```
print("Linear Kernel Accuracy:")
print(accuracy_score(y_test, y_pred_linear))
```

```
Linear Kernel Accuracy:
1.0
```

```
print(classification_report(y_test, y_pred_linear))
```

```
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       1.00      1.00      1.00         9
           2       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```

```
rbf_svm = SVC(kernel='rbf')
rbf_svm.fit(X_train, y_train)
```

```
▾ SVC
SVC()
```

```
y_pred_rbf = rbf_svm.predict(X_test)
```

```
print("RBF Kernel Accuracy:")
print(accuracy_score(y_test, y_pred_rbf))
```

```
RBF Kernel Accuracy:
1.0
```

```
poly_svm = SVC(
    kernel='poly',
    degree=3
)
poly_svm.fit(X_train, y_train)
```

```
▾ SVC
SVC(kernel='poly')
```

```
y_pred_poly = poly_svm.predict(X_test)
```

```
print("Polynomial Kernel Accuracy:")
print(accuracy_score(y_test, y_pred_poly))
```

```
Polynomial Kernel Accuracy:
1.0
```

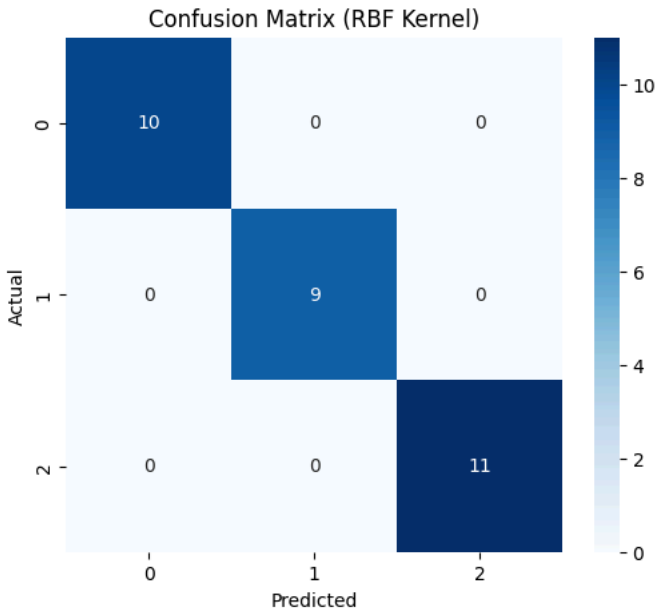
```
print(classification_report(y_test, y_pred_poly))
```

```
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       1.00      1.00      1.00         9
           2       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```

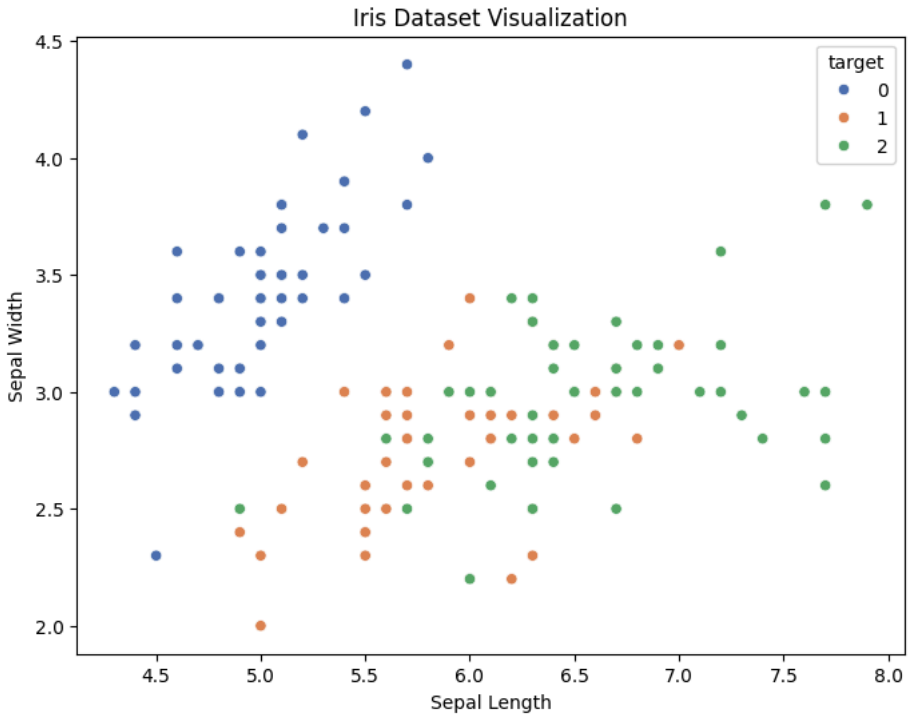
```
cm = confusion_matrix(y_test, y_pred_rbf)
print(cm)
```

```
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

```
plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title("Confusion Matrix (RBF Kernel)")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```




```
plt.figure(figsize=(8,6))

sns.scatterplot(
    x=data.iloc[:, 0],
    y=data.iloc[:, 1],
    hue=data["target"],
    palette="deep"
)

plt.title("Iris Dataset Visualization")
plt.xlabel("Sepal Length")
plt.ylabel("Sepal Width")
plt.show()
```


Iris Dataset Visualization
target
0
1
2
Sepal Width
Sepal Length
4.5
4.0
3.5
3.0
2.5
2.0
4.5
5.0
5.5
6.0
6.5
7.0
7.5
8.0